

Progetto di Comunicazione Embedded

# Sistema di controllo remoto di un drone

Integrazione hardware e multiprotocollo con STM32, ESP32, modulo USB Host CH376 e controller DualSense.

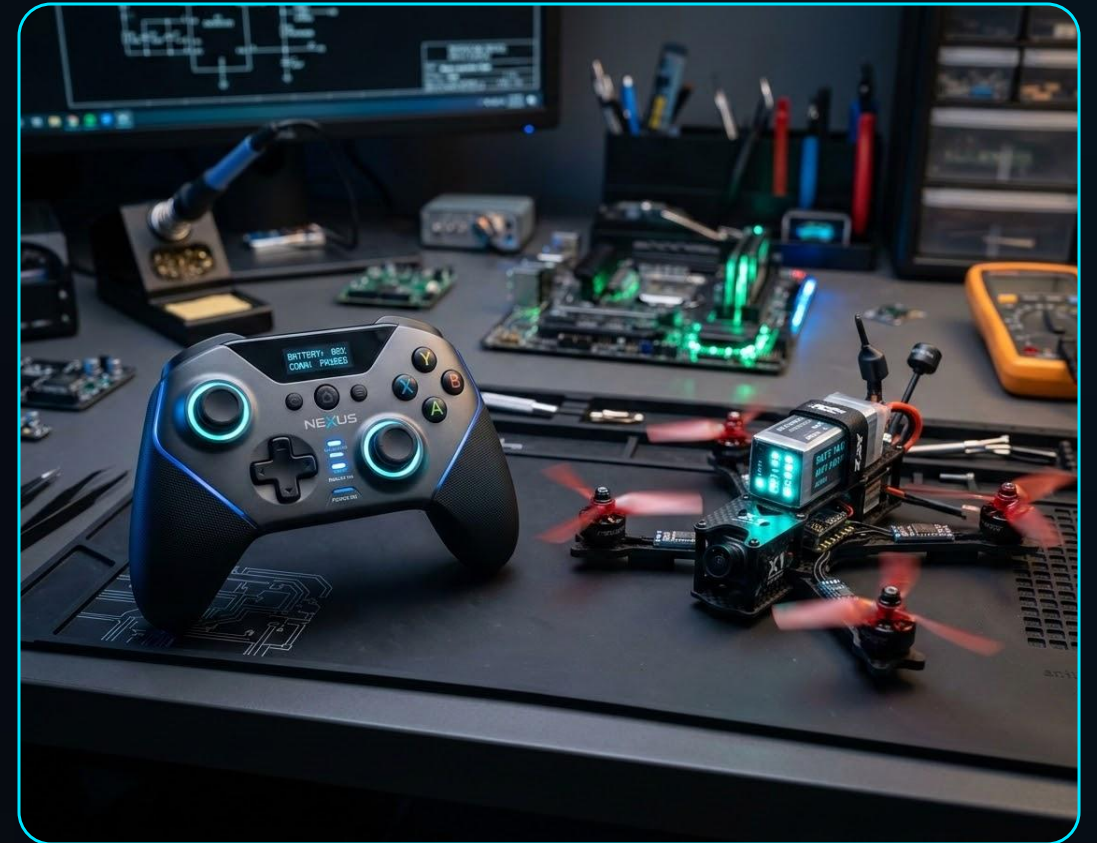


# L'Idea del Sistema

## Rivoluzionare il Controllo Remoto

L'obiettivo del progetto è realizzare un sistema di comunicazione alternativo per il pilotaggio di un drone, sostituendo i radiocomandi tradizionali con un'interfaccia differente.

Utilizzo di un **controller noto** per l'utente finale, Dualsense nello specifico.



# Panoramica e Architettura



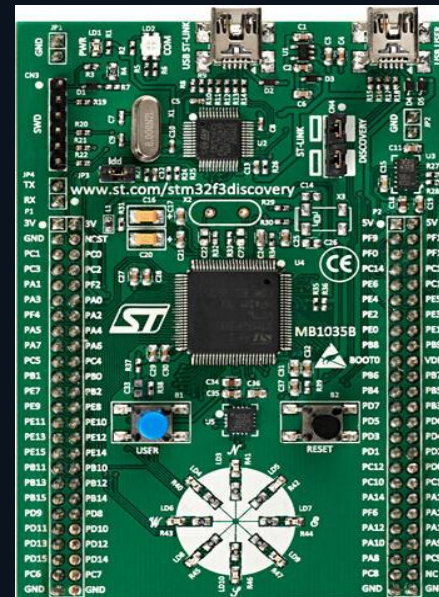
**DualSense**

Controller di Input



**CH376**

Modulo USB Host



**STM32**

Microcontrollore Principale



**ESP32**

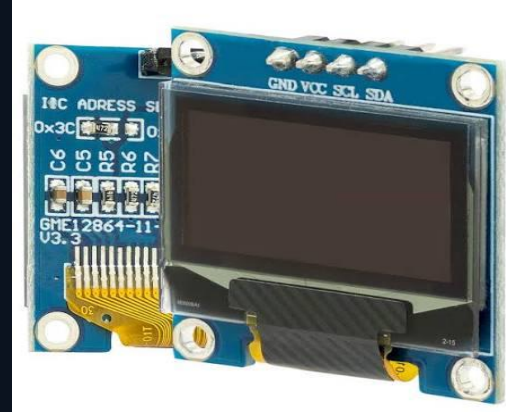
Modulo di Trasmissione

# Panoramica e Architettura



**NMY N300**

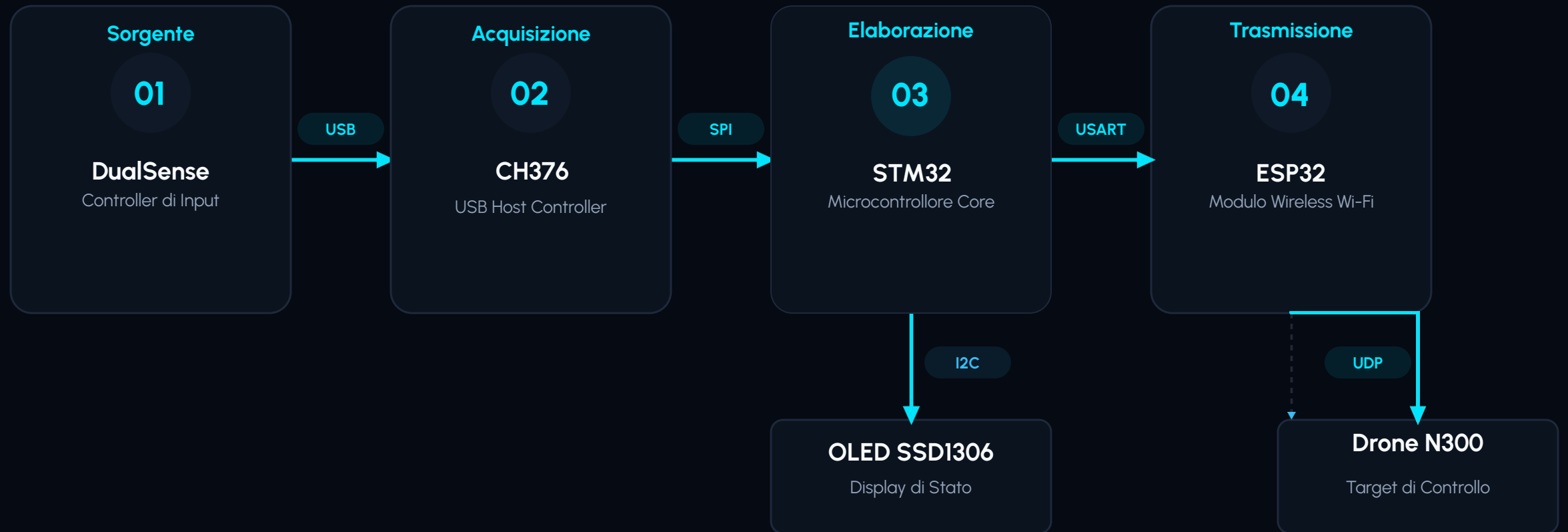
Drone di volo



**SSD1306**

Display OLED

# Architettura del Flusso dei Dati





# Fase 1: Sorgente e acquisizione input

## 1. Protocollo USB (CH376)

Il DualSense è connesso fisicamente al modulo CH376, che agisce da **USB Host**. Una Powerbank esterna garantisce l'alimentazione stabile per gestire l'assorbimento di corrente del controller PS5 durante la trasmissione dati.

Per il modulo CH376 è stato scritto un driver che svolge l'operazione di enumerazione tipica del protocollo USB, questo permette l'accoppiamento o conoscenza fra i due dispositivi. Tale fase di enumerazione è stata modellata come una **macchina a stati**.

```
uint8_t ch376_enumerateDevice(SPI_HandleTypeDef *hspi1) {
    uint8_t status = 0x00;
    uint16_t total_lenght = 0x0000;
    uint8_t config_value = 0x00;
    usb_state = USB_RESET;

    // FASE 1 --> RESET
    if (usb_state == USB_RESET){
        status = ch376_init(hspi1);
        if (status != USB_INT_SUCCESS) return status;
        usb_state = USB_RESET_HOST;
    }

    // FASE 2 --> RESET_HOST
    if (usb_state == USB_RESET_HOST){
        status = ch376_reset_host(hspi1);
        if (status != USB_INT_SUCCESS) return status;
        usb_state = USB_CONNECTION;
    }

    // FASE 3 --> CONNECTION
    if (usb_state == USB_CONNECTION){
        status = ch376_connection(hspi1);
        if (status != USB_INT_CONNECT) return status;
        usb_state = USB_GET_DEVICE_DESCRIPTOR;
    }

    // FASE 4 --> GET_DEVICE_DESCRIPTOR
    if (usb_state == USB_GET_DEVICE_DESCRIPTOR){
        status = ch376_get_descriptor(hspi1);
        if (status != USB_INT_SUCCESS) return status;
        usb_state = USB_SET_ADDRESS;
    }

    // FASE 5 --> USB_SET_ADDRESS
    if (usb_state == USB_SET_ADDRESS){
        status = ch376_setAddress(hspi1);
        if (status != USB_INT_SUCCESS) return status;
        usb_state = USB_GET_CONFIG_DESCRIPTOR;
    }
}
```

```
// FASE 6 --> USB_GET_CONFIG_DESCRIPTOR
if (usb_state == USB_GET_CONFIG_DESCRIPTOR){
    status = ch376_getConfig_Desc(hspi1,&total_lenght);
    if (status != USB_INT_SUCCESS) return status;
    usb_state = USB_GET_FULL_DESCRIPTOR;
}

// FASE 7 --> USB_GET_FULL_DESCRIPTOR
if (usb_state == USB_GET_FULL_DESCRIPTOR){
    status = ch376_GetFullDescriptor(hspi1, total_lenght, &config_value);
    if (status != USB_INT_SUCCESS) return status;
    usb_state = USB_PARSING_CFG;
}

// FASE 8 --> USB_SET_CONFIGURATION
if (usb_state == USB_PARSING_CFG){
    status = ch376_parseCfg(total_lenght);
    if (status != USB_INT_SUCCESS) return status;
    usb_state = USB_SET_CONFIGURATION;
}

// FASE 9 --> USB_PARSING_CFG
if (usb_state == USB_SET_CONFIGURATION){
    status = ch376_setConfig(hspi1, config_value);
    if (status != USB_INT_SUCCESS) return status;
    usb_state = USB_HID_GET_REPORT;
}

// FASE 10 --> USB_HID_GET_REPORT
if (usb_state == USB_HID_GET_REPORT){
    status = ch376_getHIDreport(hspi1);
    if (status != USB_INT_SUCCESS) return status;
    usb_state = USB_DONE;
}

if(usb_state == USB_DONE){
    usb_state = USB_RESET;
    return USB_INT_SUCCESS;
}
```

# Fase 2: Trasmissione CH376-STMicro32

## Protocollo SPI (CH376-STMicro32)

I dati estrapolati dal Dualsense dal CH376 dopo la fase di enumerazione vengono inviati alla STMicro32 tramite protocollo SPI. La scelta del **protocollo SPI** risiede nella gran velocità di trasmissione e nella semplicità di utilizzo di quest'ultimo, inoltre l'obiettivo è quello di avere una latenza quanto più bassa possibile per avere i dati pronti all'utilizzo.

```
// Funzione chiamata dentro takeUSBdata() quando arriva l'interrupt
uint8_t ch376_readInterruptDataAsync(SPI_HandleTypeDef *hspi, uint8_t *buffer, uint8_t *len) {
    uint8_t status;

    // Leggiamo lo stato dell'interrupt appena arrivato (questo pulisce anche il pin INT del CH376)
    ch376_sendCommand(GET_STATUS, hspi);
    status = ch376_readData(hspi);
    ch376_endCommand();

    if (status == USB_INT_SUCCESS) {
        ch376_sendCommand(RD_USB_DATA0, hspi);
        *len = ch376_readData(hspi);
        if (*len > 64){
            *len = 64;
        }
        for (int i = 0; i < (*len); i++) {
            buffer[i] = ch376_readData(hspi);
        }
        ch376_endCommand();
    }

    return status;
}
```

# Fase 3: Manipolazione dati

## Elaborazione (STM32)

La scheda agisce da Master del sistema. Oltre a smistare i dati verso la ESP32, esegue il mapping dei valori analogici del DualSense. Inoltre la STM32 provvede alla costruzione del pacchetto da inviare alla ESP32 nonché all'invio dei dati, precedentemente acquisiti dal controller, ad un display **OLED SSD1306** tramite **protocollo I2C** così da poter ottenere un feedback grafico relativamente alla modalità di utilizzo del drone, ai valori di pitch, roll, yaw e relativo anche all'attivazione dei motori.

```
void Char_SSD1306(uint8_t riga, uint8_t colonna, uint8_t *buffer, char lettera){
    uint8_t indice_font = lettera - 32;
    uint16_t posizione = (riga * 128) + colonna;

    if (lettera < 32 || lettera > 127){
        lettera = '?';
    }

    if(riga > 7 || colonna > 122) return;

    for(short int i = 0; i < CHARS_COLS_LENGTH; i++){
        buffer[posizione + i] = FONTS[indice_font][i];
    }
    buffer[posizione + 5] = 0x00; // Ultima colonna lasciamo a 0
}

void String_SSD1306(uint8_t riga, uint8_t colonna, uint8_t *buffer, const char *stringa){
    short int i = 0;
    while (stringa[i] != '\0')
    {
        if((colonna + 6) > 128){
            break;
        }
        Char_SSD1306(riga,colonna + (i*6), buffer, stringa[i]);
        i++;
    }
}
```



# Fase 4: Comunicazione ESP32

## Comunicazione ESP32

A seguito della costruzione del pacchetto la STM32 invierà il dato formattato tramite **protocollo USART**, la scelta di quest'ultimo risiede nella volontà di mantenere la consistenza e la sicurezza dei dati inviati cedendo la velocità che poteva offrire la SPI, questo è dato dal fatto che la ESP non invierà pacchetti in flooding al drone per cui sarà tempificato l'invio.

```
if (USB_allarm == 1){

    takeUSBdata();

    USB_allarm = 0;

    if(CQ_pop(&QueueDualsense,&dsd)){
        uint8_t ext_code = composePacket(&dsd,&dps);
        CQ_push(&QueueSendData,&dps);
        CQ_push(&QueueDisplayController,&dsd);
        ssd1306_pending = 1;
    }

    if(CQ_pop(&QueueSendData,&dps)){
        uint16_t idx_len = serializePacket(&dps,pData);
        if (huart1.gState == HAL_UART_STATE_READY) {
            HAL_UART_Transmit_IT(&huart1,pData,idx_len);
        }
    }
}
```

# Fase 5: ESP32 Protocollo UDP

## Comunicazione UDP con drone

La ESP32 ha ottenuto il pacchetto tramite protocollo USART, adesso tale pacchetto dovrà essere inviato al drone tramite protocollo UDP. Queste operazioni sono state svolte utilizzando **FreeRTOS**. Abbiamo creato istanziato la comunicazione con la rete Wi-Fi creata dal drone utilizzando il canale a 5GHz tipico di questo drone, a seguito di queste fasi abbiamo creato task multipli, uno per USART, uno per la gestione del LED, uno per la creazione della socket UDP.

```
static msg_t take_data(QueueHandle_t queue){ //RX
    msg_t msg;
    msg.len = 0;
    if(xQueueReceive(queue,&msg,0) == pdPASS){
        if (msg.buffer[3] == ENGINE || msg.buffer[3] == MODE || msg.buffer[3] == FLY_ON){
            ESP_LOGW("MESSAGGIO SEND", "messaggio in arrivo: %X", msg.buffer[3]);
            pck_lock = 0;
        }
        return msg;
    }
    return msg;
}

static void udp_send_task(void *pvParameters){
    udp_datas_t * data = (udp_datas_t *)pvParameters;
    int sock = data->sock;
    struct sockaddr_in dest_addr = data->dest;
    QueueHandle_t queue = data->queue;

    while (1) {
        msg_t msg = take_data(queue);
        ESP_LOGI("SOCK_SEND","msg len: %u",msg.len);
        if (msg.len > 0){
            /* for (size_t i = 0; i < msg.len; i++) {
                ESP_LOGI("SOCK_SEND: ", "%02X ", msg.buffer[i]);
            } */
            int err = sendto(sock, msg.buffer, msg.len, 0, (struct sockaddr *)&dest_addr, sizeof(dest_addr));
            if (err < 0) {
                ESP_LOGE("Sock send", "Error occurred during sending: errno %d", errno);
            } else {
                //ESP_LOGI("Sock send", "Message sent");
            }
        } else {
            ESP_LOGE("Sock send","Dato corrotto o coda vuota");
        }

        vTaskDelay(30);
    }

    if (sock != -1) {
        ESP_LOGE("Sock send", "Shutting down socket and restarting...");
        shutdown(sock, 0);
        close(sock);
    }

    vTaskDelete(NULL);
}
```

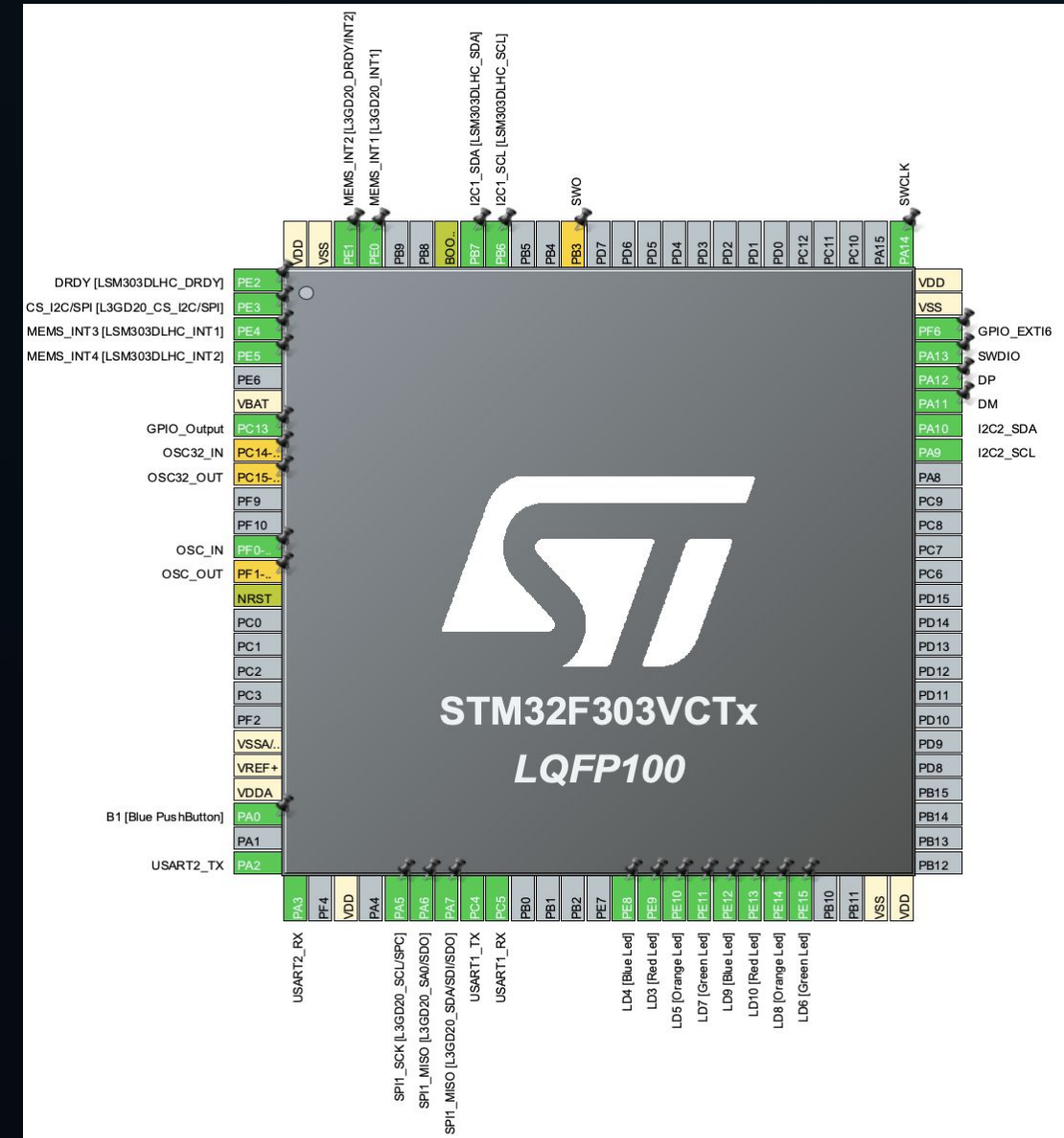
# Configurazione Hardware STM32

# Configurazione CH376-STM32

Per gestire l'arrivo dei dati dal DualseNSE abbiamo deciso di utilizzare il meccanismo dell'interrupt esterna generata dal chip CH376. Il **PIN** che svolgerà questo ruolo è **PF6** che come si vede dallo schema è gestito come **GPIO\_EXTI6**. L'uso dell'interruzione messa insieme al meccanismo di priorità implementato tramite NVIC, assegnando priorità massima ci permette di avere la massima reattività all'arrivo dei dati come da noi pensato.

## DMA per SSD1306

Sul display che ricordiamo faceva uso del protocollo **I2C** abbiamo deciso di utilizzare il **DMA**, tale scelta è motivata dalla quantità di dati da trasferire alla periferica tramite I2C e sapendo che tale protocollo non è molto veloce, per permettere alla CPU di concentrarsi sul resto del programma abbiamo deciso appunto di introdurre tale componente che trasferirà un ammontare di 1024 bytes.



# Interruzioni e gestione tramite NVIC



## IDEA

Per ottimizzare l'efficienza del firmware su STM32, i protocolli di comunicazione **SPI**, **USART** e **I2C** sono stati interamente gestiti tramite un **meccanismo di priorità**. Sfruttando il controller **NVIC**, abbiamo strutturato le priorità per prevenire conflitti, assicurando massima reattività alle linee dati critiche e delegando le operazioni pesanti in background.

### SPI (Priorità Massima)

#### Priorità NVIC: Livello 0

Gestita tramite l'interruzione esterna generata dal chip CH376 sul pin **PF6** (configurato come **GPIO\_EXTI6**).

Il Dualsense è un dispositivo veloce e mantenere la reattività massima sui dati è stato il nostro obiettivo.

### USART (Priorità Media)

#### Priorità NVIC: Livello 1

Configurata con una priorità leggermente inferiore per gestire lo scambio dati verso il modulo ESP32.

Inviare i dati è essenziale ma non reputiamo fondamentale inviare continuamente dati alla ESP poiché quest'ultima non invia dati in flooding.

### I2C / DMA (Priorità Bassa)

#### Priorità NVIC: Livello 2

Utilizzato per il trasferimento massivo di 1024 byte verso il display OLED SSD1306 tramite **DMA**.

Reputiamo il display un extra del progetto nonché un ottimo metodo per svolgere debug, anche se il dato non è il più recente non inficia le prestazioni.



# Sviluppi Futuri



Info sul drone:

[https://github.com/RubikGio/DebugDroneNMY\\_N300](https://github.com/RubikGio/DebugDroneNMY_N300)



Il progetto presenta un'architettura modulare altamente scalabile. Per incrementare l'autonomia operativa e l'esperienza d'uso, sono stati definiti tre pilastri chiave di sviluppo volti ad estendere le capacità del velivolo, la connettività e il feedback in tempo reale.



## Funzionalità Drone

### Scalabilità Hardware

Ampliare le capacità del firmware per supportare **nuovi sensori** di bordo ed estendere le logiche di volo.

- Integrazione di moduli **GPS / LiDAR** per mantenere il volo stazionario in autonomia.
- Ottimizzazione delle routine PID per aumentare la stabilità in caso di vento.



## Modulo Bluetooth

### Connessione Wireless

Rimuovere la necessità di chip hardware USB per accoppiare il controller direttamente alla scheda.

- Integrazione di un **modulo Bluetooth** per interfacciare il DualSense nativamente.
- Riduzione del cablaggio fisico, diminuendo peso e punti di fallimento sul Master.



## Sistema Telemetria

### Visualizzazione Dati

Strutturare un feedback a terra esteso tramite l'aggiunta di un secondo display di controllo.

- Implementazione di un **display aggiuntivo** per stampare la telemetria in tempo reale.
- Monitoraggio costante di: tensione batteria, stato di connessione e coordinate.